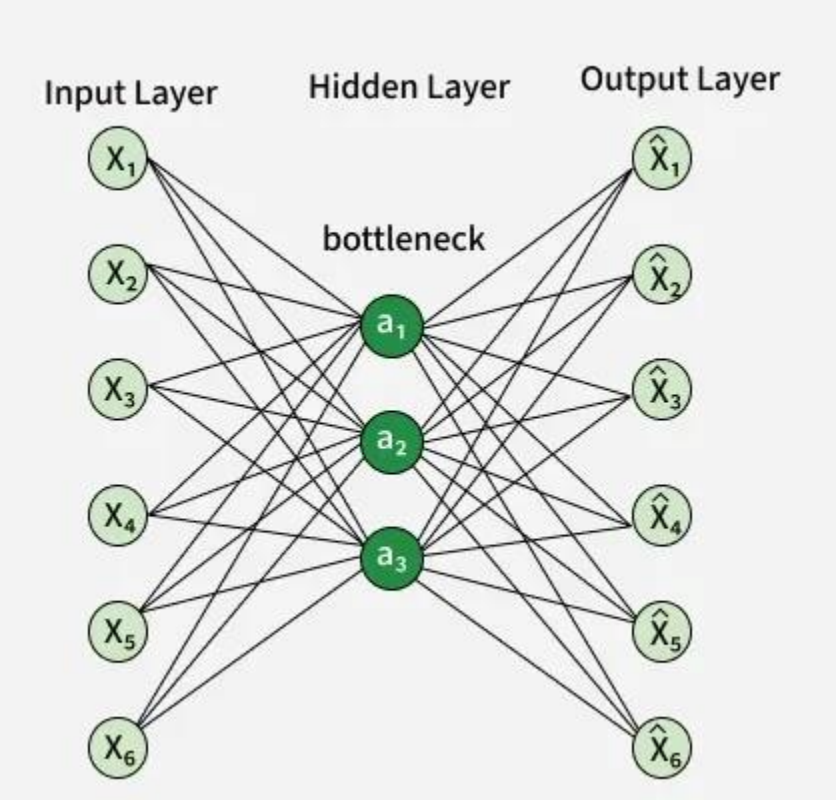
Module 3

Q. Autoencoder architecture

=>



Autoencoder Architecture

Introduction

1. An autoencoder is a type of neural network used to learn compressed representations of data.
2. The main goal of an autoencoder is to reconstruct the input data at the output layer.
3. The network learns important features automatically without using labeled data.
4. Autoencoders are mainly used for dimensionality reduction, noise removal, and feature extraction.
5. The architecture of an autoencoder consists of encoder, bottleneck, and decoder parts.

Main Components of Autoencoder Architecture

1. Input Layer

1. The input layer receives the original data.
2. The input can be image pixels, audio signals, text features, or numerical values.
3. Example: A 28 × 28 image can be given as input to the autoencoder.

2. Encoder

1. The encoder compresses the input data into a smaller representation.
2. The encoder contains one or more hidden layers.

3. The number of neurons gradually decreases in the encoder.
4. The encoder extracts only the important features of the input.
5. The encoder converts high-dimensional input into low-dimensional representation.

3. Bottleneck Layer

1. The bottleneck layer is the middle layer of the autoencoder.
2. The bottleneck layer contains the compressed form of the input data.
3. The bottleneck layer has fewer neurons than the input layer.
4. The network is forced to store only the most important information in this layer.
5. The bottleneck layer is also called latent space or code layer.

4. Decoder

1. The decoder reconstructs the original input from the compressed representation.
2. The decoder structure is usually opposite to the encoder structure.
3. The number of neurons gradually increases in the decoder.
4. The decoder tries to generate output similar to the original input.

5. Output Layer

1. The output layer produces the reconstructed data.
2. The output size is usually the same as the input size.
3. The network compares the output with the original input to calculate reconstruction error.

Working of Autoencoder

1. The input data is passed to the encoder.
2. The encoder compresses the data into a smaller form.
3. The bottleneck layer stores the compressed representation.
4. The decoder reconstructs the original data from the compressed form.
5. The reconstructed output is compared with the original input.
6. The network adjusts weights to reduce reconstruction error.

Characteristics of Autoencoder

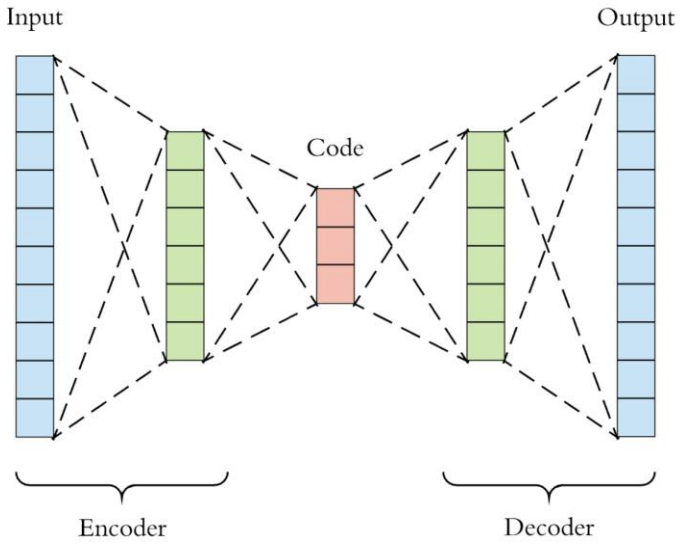
1. Autoencoders are trained using unsupervised learning.
2. Autoencoders learn efficient feature representations.
3. Autoencoders reduce dimensionality of data.
4. Autoencoders can remove noise from data.

Applications

1. Autoencoders are used for image compression.
2. Autoencoders are used for removing noise from images.
3. Autoencoders are used for anomaly detection.

Q. Linear Encoder

=>



Introduction

1. A linear encoder is a part of an autoencoder that compresses input data using only linear operations.
2. A linear encoder does not use non-linear activation functions like ReLU or Sigmoid.
3. The encoder transforms high-dimensional input into a lower-dimensional representation.
4. The main purpose of a linear encoder is dimensionality reduction.
5. A linear encoder behaves similarly to Principal Component Analysis (PCA).

Working of Linear Encoder

1. The input data is multiplied by weights and added with bias.
2. The mathematical formula is ($z = Wx + b$).
3. The symbol (x) represents the input vector.
4. The symbol (W) represents the weight matrix.
5. The symbol (b) represents the bias value.
6. The output (z) is the compressed representation of the input.
7. Since no non-linear activation function is used, the transformation remains linear.

Architecture of Linear Encoder

1. The linear encoder contains an input layer and one hidden layer.
2. The hidden layer has fewer neurons than the input layer.
3. The hidden layer acts as a bottleneck layer.
4. The encoder compresses the original data into fewer dimensions.
5. Example: An input of 100 features may be reduced to 10 features.

Characteristics of Linear Encoder

1. Linear encoder uses only linear combinations of input features.
2. Linear encoder is simple and easy to train.
3. Linear encoder is suitable for linearly related data.
4. Linear encoder cannot capture complex non-linear relationships.
5. Linear encoder gives similar results to PCA when mean squared error is used.

Applications

1. Linear encoder is used for dimensionality reduction.
2. Linear encoder is used for data compression.
3. Linear encoder is used for feature extraction.
4. Linear encoder is used in preprocessing before classification tasks.

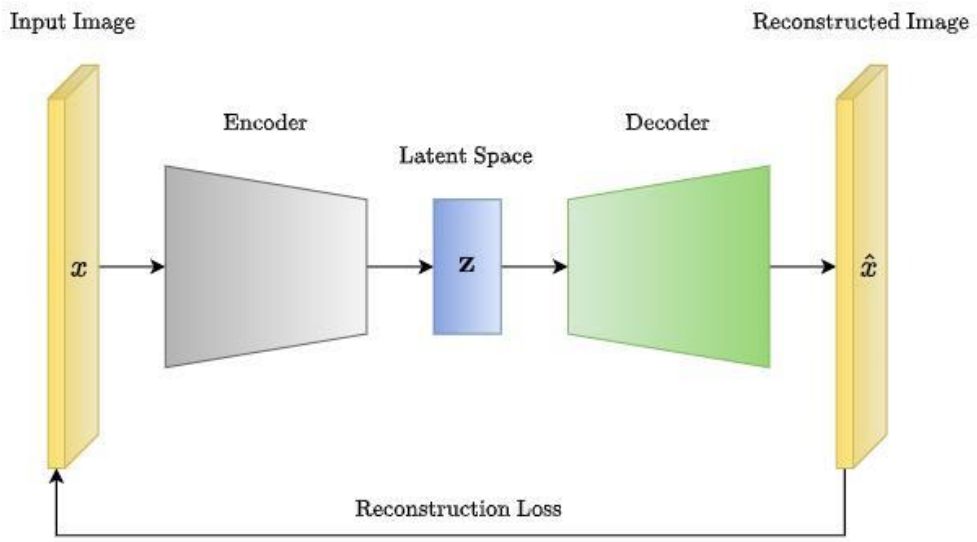
Q. Linear Encoder vs PCA

=>

Feature	Linear Encoder	PCA
Full Form	Linear Encoder in Autoencoder	Principal Component Analysis
Type	Neural network based dimensionality reduction method	Statistical dimensionality reduction method
Working Principle	Uses weights and biases to compress data	Uses principal components to reduce dimensions
Formula	Uses (z = Wx + b)	Uses covariance matrix and eigenvectors
Learning Method	Learns through training process	Does not require training
Activation Function	Usually no activation function is used	No activation function is used
Output Representation	Produces compressed hidden representation	Produces principal components
Relationship with Data	Can learn from data iteratively	Directly computes from data statistics
Complexity	Slightly more complex	Simpler than linear encoder
Training Time	Requires training time	Faster because no training is needed
Flexibility	Can be extended to non-linear autoencoders	Limited to linear dimensionality reduction
Reconstruction Ability	Can reconstruct input data through decoder	Reconstruction is possible using principal components
Handling Large Data	Suitable for large datasets with neural network training	Suitable for moderate size datasets
Accuracy	May give better results with enough training	Gives optimal linear dimensionality reduction
Main Use	Feature learning and compression	Feature extraction and data visualization
Example	Compressing image pixels using autoencoder	Reducing 100 features to 10 principal components

Q. Loss Function and Activation Function with Respect to Autoencoder

=>



Loss Function in Autoencoder

1. The loss function in an autoencoder measures the difference between the original input and the reconstructed output.
2. The main goal of the autoencoder is to reduce this reconstruction error.
3. A smaller loss value means the reconstructed output is closer to the original input.
4. The loss function helps the network learn better compressed representations.

Common Loss Functions Used in Autoencoder

1. Mean Squared Error (MSE)

1. Mean Squared Error is commonly used when the input values are continuous.
2. MSE calculates the average squared difference between original input and reconstructed output.
3. The formula is

$$MSE = \frac{1}{n}\sum(x - \hat{x})^2$$

4. The symbol (x) represents the original input.
5. The symbol (\hat{x}) represents the reconstructed output.
6. MSE is mainly used for grayscale images and numerical data.

2. Binary Cross Entropy (BCE)

1. Binary Cross Entropy is used when input values are binary or normalized between 0 and 1.
2. BCE measures the difference between actual and predicted probabilities.
3. BCE is commonly used for black-and-white images or binary features.
4. BCE is often used with Sigmoid activation in the output layer.

3. Cross Entropy Loss

1. Cross Entropy Loss is used when the output represents multiple classes.
2. This loss is less common in simple autoencoders.
3. This loss is mainly used in classification-based autoencoders.

Activation Functions in Autoencoder

1. Activation functions are used in encoder and decoder layers to introduce non-linearity.
2. Activation functions help the autoencoder learn complex patterns in the data.
3. Different activation functions are used in hidden layers and output layers.

Common Activation Functions Used in Autoencoder

1. ReLU Activation

1. ReLU is widely used in hidden layers of autoencoders.
2. ReLU gives output 0 for negative values and same value for positive values.
3. ReLU helps in faster training.
4. ReLU reduces the vanishing gradient problem.

2. Sigmoid Activation

1. Sigmoid gives output values between 0 and 1.
2. Sigmoid is commonly used in the output layer when the input data is normalized.
3. Sigmoid is suitable for binary images and probability values.

3. Tanh Activation

1. Tanh gives output values between −1 and +1.
2. Tanh is useful when the data contains both positive and negative values.
3. Tanh is sometimes used in hidden layers.

4. Linear Activation

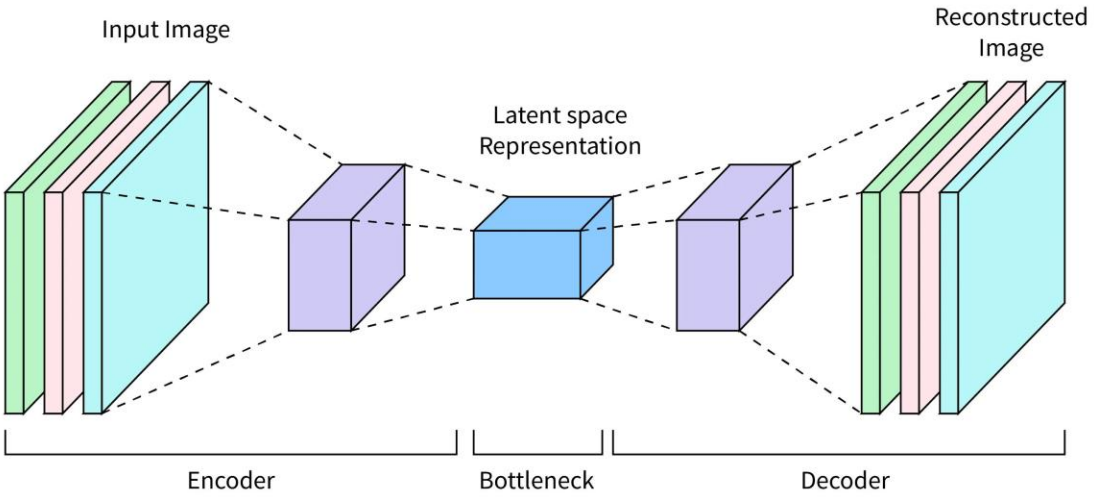
1. Linear activation gives output equal to input.
2. Linear activation is used when the autoencoder deals with continuous values.
3. Linear activation is useful in linear autoencoders.

Common Combinations

Data Type	Output Activation Function	Loss Function
Continuous numerical data	Linear	Mean Squared Error
Normalized image data (0 to 1)	Sigmoid	Binary Cross Entropy
Data with negative and positive values	Tanh	Mean Squared Error
Binary data	Sigmoid	Binary Cross Entropy

Q. Autoencoder for Image Compression

=>



SCALER Topics

Introduction

1. An autoencoder can be used for image compression by reducing the size of image data.
2. The autoencoder learns how to represent an image using fewer values.
3. The compressed representation is stored in the bottleneck layer.
4. The decoder reconstructs the image from the compressed data.
5. The reconstructed image is similar to the original image.

Working of Autoencoder for Image Compression

1. The input image is given to the encoder.
2. The encoder extracts important features from the image.
3. The encoder reduces the number of features step by step.
4. The bottleneck layer stores the compressed representation of the image.
5. The compressed representation requires less storage space.
6. The decoder reconstructs the original image from the compressed representation.
7. The output image is compared with the original image.
8. The network adjusts weights to reduce reconstruction error.

Architecture Used for Image Compression

1. The input layer receives the image pixels.
2. The encoder contains multiple hidden layers with decreasing number of neurons.
3. The bottleneck layer contains very few neurons.

4. The decoder contains hidden layers with increasing number of neurons.
5. The output layer reconstructs the original image size.

Example

1. Suppose an image contains 1000 pixel values.
2. The encoder may compress these 1000 values into only 100 values.
3. These 100 values are stored in the bottleneck layer.
4. The decoder uses these 100 values to reconstruct the image.
5. This process reduces storage space while keeping important image details.